

Uncovering Flaws in Network Protocol Implementations Using Differential Symbolic Execution

Hooman Asadian
hooman.asadian@it.uu.se
Uppsala University
Uppsala, Sweden

Bengt Jonsson
bengt@it.uu.se
Uppsala University
Uppsala, Sweden

Paul Fiterău-Broștean
paul.fiterau_brostean@it.uu.se
Uppsala University
Uppsala, Sweden

Konstantinos Sagonas
kostis@it.uu.se
Uppsala University
Uppsala, Sweden
National Technical University of Athens
Athens, Greece

ABSTRACT

Implementations of network protocols must closely follow their specifications to avoid security flaws and interoperability issues. Ensuring compliance is difficult given the complex and often stateful nature of modern protocols. Oracle-driven, specification-based testing helps but checks only requirements that have been explicitly encoded, which leaves gaps for complex, stateful protocols such as DTLS. We present a differential symbolic execution approach that removes reliance on an explicitly specified oracle by comparing implementations on the same interaction while exploring classes of inputs. Our approach symbolically executes a common harness for each system under test, observes the packet exchange, tracks interaction state, makes selected input fields symbolic, and records compact, semantically meaningful outputs for every explored path. For pairs of tested implementations, we use SMT solving to identify pairs of paths which can be exercised by the same input, but result in different outputs. We then check the implementations' divergent behaviors for compliance to the specification. We applied our technique to the most recent versions of several widely used DTLS implementations and detected nine distinct bugs across versions, including five that were previously unknown and were later fixed or confirmed.

1 INTRODUCTION

Testing network protocol implementations against their specifications is essential for ensuring secure, reliable, and interoperable communication. Even minor deviations from the protocol specification can cause interoperability failures or introduce subtle security vulnerabilities that attackers can exploit. For example, in the case of the TLS protocol, seemingly trivial errors in handling input fields have led to serious attacks such as ROBOT [4] and the POODLE downgrade attack [3, 23].

Several established techniques exist for testing protocol implementations. A prominent approach is specification-based testing [18, 19, 22], where tests are derived directly from the official protocol specification (for example, an RFC). The idea is to execute the implementation with selected inputs and check the corresponding outputs against the requirements expressed in the specification. In practice, this often involves creating an oracle that encodes the

requirements, e.g., in the form of assertions, to detect deviations from the specification.

The detection power in specification-based testing can be significantly enhanced by using Symbolic Execution [20] (SE), which explores an implementation's response to the full range of possible inputs, rather than just a few selected ones. Symbolic execution is a white-box technique that analyzes programs in which some inputs or variables are designated as *symbolic*, and explores the code paths that are feasible for some assignment to these symbolic inputs, thereby reasoning over the entire space of their possible input values. In the context of protocol testing, SE can be used to explore a wide range of behaviors exhibited by an implementation. These behaviors are then checked, which has traditionally relied on an oracle incorporating checks for selected requirements [1, 2, 29–31]. We call such an approach *specification-based testing*.

While specification-based testing is effective for cases where the requirements can be accurately captured in an oracle, it is inherently incomplete: any requirements not explicitly encoded in the oracle will not be checked. For complex protocols such as TLS and DTLS, whose specifications are long, intricate, and sometimes ambiguous, manually encoding all requirements and corner cases is impractical, leaving coverage gaps where bugs can hide. Even if enhanced by symbolic execution, ensuring compliance still depends on the oracle; if a particular requirement or edge case is missing from the oracle, violations in that area will remain undetected.

Differential testing removes the need for specification-based oracles by comparing the behaviors of multiple implementations. The idea, originating from McKeeman's work [21], is to compare multiple independent implementations of the same protocol on identical inputs and flag any differences as potential flaws. This method has proven highly effective in uncovering bugs, as demonstrated by e.g., the "Frankencerts" study on TLS certificate validation [5] and the differential fuzzing of C compilers [35]. Differential testing can be further enhanced by integrating SE, a technique known as *differential symbolic execution*, which explores a broader range of behaviors to detect subtle differences. This approach has been successfully applied to detect inconsistencies in protocol message parsing logic [36] and to identify noncompliance in X.509 certificate validation implementations [7]. However, it has so far only

been employed to test *stateless* components of network protocol implementations.

In this work, we employ differential symbolic execution to test the *stateful* behavior of network protocol implementation. Specifically, we exercise multiple implementations on the same concrete protocol interaction while selectively making chosen input fields symbolic. SE then explores paths exercised on each implementation using different values for the symbolic fields. For each path, we record the path condition and a compact set of semantically-meaningful outputs. We then check whether there exists an input that satisfies two recorded path conditions for different implementations while the corresponding outputs differ. When such an input exists, we consult the RFC to determine whether the difference reflects non-compliance or a security-relevant flaw. By generating differences automatically, our approach can expose non-conforming behaviors even with respect to corner-case requirements that are easily missed by specification-based testing.

Our approach proved highly effective during this study: we analyzed four popular DTLS implementations and automatically identified numerous discrepancies resulting in the discovery of nine bugs, of which five were previously unknown. Furthermore, we demonstrate that our approach can be applied in a cross-version setting (differentially testing an implementation against a prior version of itself) to catch regressions.

In summary, our work:

- Presents a differential symbolic execution method for testing stateful behavior of network protocol implementations without requiring an explicitly specified oracle to guide the testing.
- Evaluates our approach on four real-world DTLS server implementations (Mbed TLS, OpenSSL, TinyDTLS, and wolfSSL). Our analysis identified multiple discrepancies in how these implementations handle edge cases leading to the discovery of nine distinct bugs, five of which were previously unknown and have since been fixed or confirmed.
- Compares our approach against a state-of-the-art approach in specification-based testing using symbolic execution. Our approach is able to find all the bugs reported by the other approach, as well as new bugs.
- Provides a publicly available infrastructure to enable the replication of our experiments or the extension of our work.

We next present our methodology for differential symbolic execution of stateful protocol implementations, including the common test harness, lightweight monitors, and discrepancy detection procedure (§2). We then describe our implementation (§3), evaluate the approach on four DTLS servers with a comparison to a specification-based SE baseline and a study of cross-version regressions (§4), review related work (§5), and conclude (§6).

2 METHODOLOGY

In this section, we present our methodology for identifying differences in the responses different protocol implementations produce for the same input. Our approach involves, for each System Under Test (SUT), first creating a common test harness (§2.1), then symbolically executing it to record what outputs the SUT produces for different values of selected fields in input packets (§2.2). We use

the result of SE to detect differences in the outputs different SUTs generate for the same input. These differences are then investigated with respect to protocol specification (§2.3). An important step in our approach involves selecting the fields (1) to make symbolic in input packets, and (2) to consider when comparing the responses of different SUTs to the same input. This step is addressed separately in §2.4. To aid understanding, this section uses a running example to show our approach can be employed to test real implementations of DTLS and find an actual bug.

2.1 Creating a Test Harness

The initial step involves constructing for each SUT a test harness which orchestrates a predefined interaction between the SUT and another protocol party (e.g., client/server). This interaction might involve completing a handshake, or more generally, exchanging a set of packets between the protocol parties. The test harness connects two components: the SUT, which is the target of the analysis, and another protocol party, called the *facilitator*, which provides the necessary inputs to drive the interaction forward. For example, in a client/server protocol such as DTLS, if the SUTs are server implementations, the facilitator will be a client implementation. Since the goal is to examine how different SUTs respond to the same inputs, for a SUT protocol role (e.g., server) we use the same facilitator (e.g., client implementation of a particular library) for all the tested SUTs (e.g., server implementations).

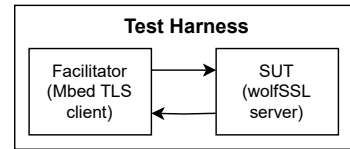


Figure 1: Architecture of the test harness used to test wolfSSL’s server implementation.

Suppose we want to find differences between two DTLS server implementations: Mbed TLS and wolfSSL. Applying our approach requires developing two separate test harnesses, one for each SUT. These test harnesses share the same facilitator. The facilitator needs to be based on a DTLS client implementation, e.g., Mbed TLS,¹ as is the case in our evaluation. Figure 1 gives a high-level view of the test harness we use to test wolfSSL.

2.2 Running Symbolic Execution

Assuming there are multiple protocol implementations available, we can already perform *basic differential testing*. Concretely, we could execute the SUT on the predefined interaction using the corresponding test harness, observe selected response packets, and flag pairwise differences. However, this evaluates only the responses to the concrete input packets in a single run, so many corner cases might remain untested. To go beyond the limits of basic differential testing, we leverage the power of symbolic execution to check the SUT’s responses to classes of input packets, thereby performing *differential SE*. This is done by making selected fields in facilitator-supplied input packets symbolic, and executing the entire harness

¹The facilitator can very well be a different library from those tested, e.g., OpenSSL.

(including the SUT and the facilitator) under an SE engine. The SE engine will then explore the code paths that are feasible for some values of these symbolic fields, and for each code path record the outputs of the SUT, including how these outputs depend on the symbolic input fields. Technically, this is achieved by recording the outputs as *symbolic expressions*. These per-path artifacts are later compared across implementations to detect discrepancies (see §2.3).

For each pair of SUTs, we run multiple differential SE campaigns. Each campaign analyzes a specific interaction and makes specific fields symbolic. In order to manage these campaigns in an orderly way, we concentrate the logic for each campaign in a lightweight *monitor*. A monitor is a software component that observes and may influence the interaction between the facilitator and the SUT (e.g., by making fields of concrete packets symbolic). It encapsulates the testing logic for a single differential SE campaign: it declares which fields to make symbolic, at which point in an interaction to make the field symbolic, and what fields to record from the SUT’s response. We adapt the monitor concept from Asadian et al. [2]. In our setting the monitor does not check requirements (as in [2]); instead, it (i) makes selected fields in input packets symbolic at chosen points in the interaction between facilitator and SUT, and (ii) records the SUT’s immediate response.²

To explain how monitors are defined and employed in our approach, we return to our running example. Let us assume we want to find discrepancies between the Mbed TLS and wolfSSL servers by considering their handling of the DTLS field `sequence_number`. This 48-bit field is present in every DTLS record (packet), uniquely identifying it. The value of `sequence_number` is incremented in each record sent, typically starting from 0. This allows the recipient to process records in the right order³, and to detect duplicated records (which have the same `sequence_number`). Testing that SUTs properly handle this field is difficult owing to its large domain, and the many requirements specified over it by the DTLS specification. An oracle used for specification-based testing can easily miss some of its corner-case requirements. As we will see, our approach can check even such requirements and find bugs doing so.

To analyze the SUT’s handling of `sequence_number`, we use a monitor which marks `sequence_number` as symbolic for each input packet where this field appears, and records the SUT’s response to that packet. We succinctly express the behavior of such monitors using a state machine formalism, as shown in Fig. 2. A monitor is represented as a state machine whose local state consists of a *control state* and possibly additional local variables. The monitor in Fig. 2 has three control states, represented by the three circles. It observes packets sent by the client or the server and may perform a transition whenever an observed packet satisfies the condition above the bar in the label of an outgoing transition. We let P be the observed packet, and let $P.source$ denote its origin. When performing the transition, the monitor executes the statement below the bar.

Starting from the initial state `Init`, the monitor in Fig. 2 can (non-deterministically) take one of two actions upon receiving a client packet. Either it takes the self-loop transition and remains in `Init`, or it transitions to `Rcvd`, making the `sequence_number` in the observed packet symbolic. Once in `Rcvd`, if the monitor observes a server

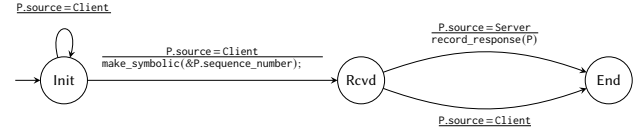


Figure 2: Monitor recording a DTLS server’s responses for different values of `sequence_number`.

packet (which the monitor sees as a response to the previous client packet), it records it and transitions to `End`. In case, however, it sees another client packet, it assumes that the server has generated no response and transitions to `End`. Arrival to `End` terminates the execution, prompting SE to record the condition of the terminated path. The self-loop in `Init` allows the monitor to transition to `Rcvd` on any packet from the client during an interaction, make the packet’s `sequence_number` symbolic and record the server’s response (if any).

To identify the packet containing the symbolic field, we also record the packet’s position in the interaction, which we refer to as the *index* (e.g., the first packet has index 0; if two packets precede it, the index is 2). We then only compare responses to packets with the same index, so the SUTs are in the same interaction state. The index is not tracked inside the monitor. Instead, our infrastructure assigns every packet a concrete counter value and introduces a *symbolic* variable that selects which packet the monitor will act on; both this selected index and the symbolic field become part of the path condition. Further details appear in §3.

Running symbolic execution on wolfSSL and Mbed TLS with the monitor in Fig. 2 produces, for each implementation, a set of explored paths. For every path, we record the path condition and the SUT’s observed response at the selected interaction point. In §2.3 we describe how these per-path records are compared across implementations to reveal discrepancies.

Finally, to limit the number of discrepancies and reduce the manual effort required to analyze them, we use a single monitor per SE campaign. The monitors we use make only one instance of a field symbolic, while all other instances of the same field (e.g., those in other packets) remain fixed to the concrete values assigned during the interaction.

2.3 Finding and Analyzing Discrepancies

Given two SUTs, our objective is to identify common inputs that prompt differing outputs (responses) between the SUTs. We do this via the three-step procedure described below.

Deriving Path-Output Sets. For each SUT, SE generates a set of Path-Output pairs, one for each path explored. A pair takes the form $\langle pc, o \rangle$ where pc represents the path condition and o represents the output generated (recorded response) when exploring the path. Both pc and o are parametric on the field made symbolic and the packet index. Let S_I denote the set of Path-Output pairs generated by the SE campaign for implementation I .

Deriving Discrepancies Between Implementations. Given the sets S_I for several implementations, we compare for every pair of implementations I and I' , the sets S_I and $S_{I'}$, to determine inputs for which I and I' produce different outputs while in the same state

²We use the term *recording responses* to mean recording selected fields of responses.

³Since DTLS runs over UDP, records can be received out-of-order.

of the interaction. If some concrete input can cause I and I' to produce different outputs, there must be a path of I characterized by a Path-Output pair $\langle pc, o \rangle$ and a path of I' characterized by a Path-Output pair $\langle pc', o' \rangle$ such that: (1) pc and pc' are satisfied by this common value of the input; and (2) this value results in different values of o and o' . We can express this by the conjunction:

$$(pc \wedge pc') \wedge (o \neq o') \quad (1)$$

Conjoining $(o \neq o')$ ensures that this common input value will prompt different outputs on I and I' . For each pair of Path-Output pairs $\langle pc, o \rangle$ and $\langle pc', o' \rangle$, we check the satisfiability of Formula (1) using an SMT solver. In case of satisfiability, we ask the solver to produce a valuation for the symbolic input field and the containing packet index, that satisfies Formula (1). We include each valuation in a (*discrepancy*) *witness* along with the differing outputs o and o' produced by the two SUTs. Our approach generates a witness for every pair of Path-Output pairs satisfying Formula (1).

Analyzing Discrepancies for Bugs. For every discrepancy, the protocol's RFC is consulted to determine whether the compared SUTs are compliant. In case one of the SUTs is not, we validate the non-conformance by executing the test harness using an unmodified SUT (the SUT can be optionally subjected to minor modifications; see §3). We recall that a discrepancy includes a value for the field that is made symbolic and the index of the packet containing the field such that, assigning this value to the field will prompt different outputs for two SUTs. During execution, we modify the packet by assigning the value to the field, and check whether the issue can be reproduced. If it can, we have identified a non-conformance, and we report it to developers.

We end this section by concluding our running example. For Mbed TLS, 35 Path-Output pairs were generated, while for wolfSSL, 28 Path-Output pairs were generated by following the steps we have described. We instantiated Formula (1) for these pairs, and found two which satisfy Formula (1), forming a discrepancy witness for each. A witness contains the value for `sequence_number` and for the index of the packet containing it, and for each SUT, its response to the input packet when its `sequence_number` is assigned the included value. One of the witnesses suggests that in the initial state, when processing the first packet, upon receiving a first *ClientHello* packet with `sequence_number` that is slightly over 2^{47} , Mbed TLS responds with *HelloVerifyRequest*, indicating that it has accepted the packet, while wolfSSL does not produce any response, appearing to silently discard the packet. Consulting the RFC, we determine that Mbed TLS has handled this packet correctly. In contrast, wolfSSL has wrongly rejected a packet with a large but not invalid `sequence_number`. This is bug in wolfSSL, which can cause interoperability issues. The second witness reflects a difference in strictness regarding reuse of a record sequence number across handshake records; although the implementations behave differently, both behaviors are permitted by the RFC. Section 4 describes this and some other bugs we identified by analyzing discrepancies.

2.4 Selecting Fields

Choosing which fields to make symbolic and which to record from response packets can greatly influence the effectiveness of our approach. In this section, we give some general guidelines.

2.4.1 Selecting Fields to Make Symbolic in Input Packets. In theory, we can test implementations on all fields, particularly those like `sequence_number`, whose incorrect handling can impact security and/or functionality. In practice, for the sake of performance, we may avoid fields fed to cryptographic functions, which are difficult to handle by symbolic execution. Accessing these fields in monitors might also require additional machinery (e.g., for decryption). One option is to consider only unencrypted fields, as they can be accessed without decrypting packets. Such is the case for `sequence_number`, which is unencrypted in DTLS records, and thus can be accessed without decryption. Another option is to disable encryption, which can be done via settings supported by the SUT, or by minor SUT modifications. For DTLS, we did the former (cf. §3.1).

2.4.2 Selecting Fields to Record in Response Packets. Similarly to input fields, we can record the entire packet to maximize the number of discrepancies we can identify. Finding many discrepancies, however, this comes with the cost of increased time for analyzing them. Hence, we want to reduce the number of fields to make it more manageable for analysis. To that end, our approach excludes from analysis three classes of fields which are very likely to produce uninteresting (e.g., unlikely to expose bugs) discrepancies. The first class comprises fields assigned nonces (i.e., random values). Recording nonces in response packets will lead to trivial discrepancies due to nonces being different across the responses of the SUTs. For example, we did not consider `random` in DTLS's *ServerHello* message, as this field stores a nonce. The second class comprises fields whose different values likely reflect differences in the features/settings supported by the SUTs which cannot be easily eliminated (e.g., by enabling only common features/settings across SUTs, or by making minor SUT code changes). Recording such fields would lead to trivial discrepancies due to varying support across SUTs. For example, we did not consider the field `cipher_suites` in DTLS's *ServerHello* message, whose value depends on the algorithms (e.g., for encryption) supported by the SUT, which vary greatly among SUTs. The final class comprises fields whose values depend on the values of fields in the first and second classes. As an example, DTLS's *Finished* message contains `verify_data`, which we did not consider as it is a hash over values of fields including `cipher_suites` and `random`.

3 IMPLEMENTATION

This section details how we implemented steps in §2 for testing DTLS server implementations. We start by describing the development of the test harness (§3.1), designed to coordinate interactions between the facilitator and the SUT. We then outline the augmentations made to the SE engine (§3.2) so that Path-Output pairs are generated for each SUT tested. Pairs generated for different SUTs are then compared to find and analyze discrepancies, which is our final step (§3.3). The section ends with an overview of the adaptations made to the SUT to optimize symbolic execution (§3.4).

3.1 Creating a Test Harness

A test harness is a single executable containing two modules: the SUT and a facilitator that acts as the peer (a client when the SUT is a server). Since KLEE [6], the symbolic execution engine we use, supports only single-threaded processes, both modules are consolidated into one single-threaded Unix process. Within this

process, the facilitator and the SUT exchange packets via an in-memory queue by overriding POSIX socket APIs; implementation details are given in §3.2.1.

Protocol implementations are typically shipped as libraries that include a set of APIs, allowing users to develop custom programs tailored to their needs. These libraries often include sample programs to demonstrate their usage and functionality. We constructed the harness by adapting these samples. In the DTLS setting, we started from the sample client and server provided by each library, and refactored them into the facilitator and SUT modules. To ensure all the SUTs are tested on similar inputs, we fixed the facilitator to a single library implementation (e.g., an Mbed TLS DTLS client) and varied only the SUTs (e.g., Mbed TLS and wolfSSL DTLS servers). Although a new harness is required for each library, our experience is that a harness can usually be reused with little or no change across different versions of the same library (see §4).

For each DTLS library, the harness performs a handshake and then exchanges a small unit of *application data* between the facilitator and the SUT. The two roles execute in a loop that terminates when the handshake has completed on both sides and exactly one record containing *application data* has been exchanged in each direction. The loop also exits on any fatal alert, or when an iteration produces no new bytes. We limit execution to at most N iterations.

3.2 Augmentations to the SE Engine

We execute the test harness with KLEE, using the engine to mark selected fields as symbolic in specific packets exchanged during the interaction and to record the indices of those packets along with the SUT’s responses. To this end, we augment KLEE with components adapted from the monitor-based framework of Asadian et al. [2].

3.2.1 Common Infrastructure. Many network protocols rely on POSIX sockets for packet exchange, leveraging APIs provided by the operating system’s library [32]. We employ as-is the generic infrastructure of Asadian et al. as proxy to this library, in order to mediate, by redirecting API calls, communication between the SUT and facilitator. We replace the standard socket APIs with custom versions linked with the harness; when the harness runs under KLEE, it calls our versions instead of the OS ones. This proxy works for any protocol that exchanges packets through POSIX sockets.

Concretely, calls that send bytes to a socket (e.g., `sendto`) are intercepted and their payloads are enqueued in an in-memory queue. Calls that receive bytes (e.g., `recvfrom`) dequeue from this queue instead. Upon receiving a packet, the proxy runs a monitor in three steps:

Parsing: The first packet in the queue is parsed by a protocol-specific parser (§3.2.2) and copied into a data structure P .

Execution: The monitor runs with P as input and makes selected fields in P symbolic.

Return: After the execution of the monitor completes, P is serialized and returned to the original caller.

Each direction of the data flow (e.g., client to server and server to client) has its own packet queue. We override `bind` to initialize these queues; other socket functions remain unchanged.

3.2.2 Protocol-Specific Infrastructure. Our implementation also includes the following components for each protocol considered:

- a set of monitors, one for every input field considered for symbolic execution (as per §2.4.1);
- an output-capture function, meant to record selected response fields (as per §2.4.1);
- a serializer and parser which are optional, but facilitate constructing monitors and the output-capture function.

While the first and the third components are adapted from Asadian et al., the output-capture function is specific to our work.

Set of Monitors. We define monitors to plug in to the common infrastructure in the same way as Asadian et al., albeit our monitors are much simpler as their purpose is simply to make fields symbolic rather than to check requirements. Monitors are implemented as C functions that accept a pointer to a data structure (P) as a parameter and access packet fields by referencing the corresponding fields within this structure.

```

1  enum STATE { INIT, RCVD, END };
2
3  static STATE curr_state = INIT;
4
5  void sequence_number_diff_testing_server(RECORD *P) {
6      if (curr_state == INIT && P->source == CLIENT) {
7          kleener_make_symbolic(&P->sequence_number,
8                               sizeof(P->sequence_number), "sequence_number");
9          curr_state = RCVD;
10     }
11     else if (curr_state == RCVD && P->source == SERVER) {
12         capture_dtls_output(P);
13         curr_state = END;
14     }
15     else if (curr_state == RCVD && P->source == CLIENT)
16         curr_state = END;
17 }

```

Figure 3: The C implementation of the monitor of Fig. 2.

Figure 3 presents the C code for implementing the monitor for `sequence_number` in Fig. 2. The code begins by declaring an enumeration for the monitor’s states and a static variable to store the monitor’s current state (lines 1 and 3). The function implementing the monitor accepts a pointer to the data structure containing the currently observed packet. Its logic is straightforward and closely follows the preconditions and actions specified in Fig. 2. The `kleener_make_symbolic()` function is a wrapper around KLEE’s `klee_make_symbolic()` and enables treating only part of a buffer as symbolic. It takes three arguments: a pointer to the relevant packet field, its size, and a descriptive name.

Our methodology needs the indices of the input packets whose fields have been made symbolic. Our code in Fig. 3 does not provide them, nor does it define the non-deterministic self-loop in the `Init` state. Both are implemented in the infrastructure within which monitors are run. This infrastructure currently supports non-determinism in the initial state between two transitions, one of which is a self-loop. It assigns each packet P an index (where the i -th packet in the interaction has index i), and a symbolic variable `trigger_index` declared to range over all possible packet indices. The infrastructure ensures that the non-self-loop transition is taken if its guard evaluates to true and the index of the current packet equals `trigger_index`. Otherwise, the self-loop transition is selected. Since `trigger_index` is symbolic, the SE engine

will explore all feasible execution paths in which the monitor performs an arbitrary number of self-loops followed by a non-self-loop transition. Each explored path corresponds to a specific value of `trigger_index`. Making `trigger_index` and the input fields symbolic makes KLEE include them in the path condition. This allows to get values for them by solving Formula (1). In each campaign, we run one monitor and mark as symbolic only one instance of the chosen field; other instances remain concrete, thereby limiting the number of distinct discrepancies produced downstream (§3.3).

Output-capture function. Output recording is implemented as a function invoked by the monitor. For DTLS, we implement an output-capture function (`capture_dtls_output(P)`) that reads the parsed response packet `P`, selects the response fields specified in §2.4.2, and makes KLEE dump their valuation in a separate file, as part of the output the output generated for the corresponding path.

When a recorded output field depends directly on symbolic inputs, the function records the corresponding symbolic expression as the value of that field. See the invocation in Fig. 3.

Parser & Serializer. These optional components simplify and make the definition of monitors more intuitive. The parser converts protocol packets into structured data, enabling monitors to access specific packet fields (e.g., `P->sequence_number` in Fig. 3). Conversely, the serializer reconstructs protocol packets from the data structures, whose fields may have been made symbolic by the monitors. Without these components, monitors would need to directly manipulate the bytes at specific offsets within the byte array representing the protocol packet, significantly increasing complexity. The parser also facilitates construction of the output-capture function in a similar way. For our application to DTLS, we reused the parser and serializer from Asadian et al. [2].

3.3 Finding and Analyzing Discrepancies

Executing the test harness with the symbolic execution engine produces a set of Path-Output pairs for each tested SUT, as described in §2.3. These pairs are then analyzed pairwise to identify discrepancies between the two respective SUTs. Concretely, for each field considered, we run our augmented KLEE on each tested SUT. This produces, for each SUT, a folder which includes for each path, a file specifying the path condition, and a file containing the response recorded on that path by our output-capture function. Both the path condition and the output are expressed in SMT-LIBv2 format. The folders generated by KLEE for every pair of SUTs are fed to a Python script, which logs discrepancy witnesses that we later analyze. For every pair of explored paths, one for each SUT compared, the script uses the aforementioned files generated for these paths to instantiate and check the satisfiability of Formula (1) using Z3. If Formula (1) is not satisfiable, our script discards the pair. Otherwise, it uses Z3 to obtain satisfying values for the symbolic variables, and build a discrepancy witness (comprising the values of symbolic input field, packet indices and differing response fields), logging it for later analysis. Since discrepancy witnesses are analyzed manually, our script can deduplicate the witnesses before printing them. Deduplication keeps only one witness for each distinct combination of `trigger_index` and the response fields. For instance, in our running example, our script identifies ten discrepancies; deduplication

reduces this number to two representative witnesses. Since each SE campaign marks as symbolic only a single field instance and we retain only deduplicated witnesses, the number of discrepancies we must inspect remains modest.

3.4 Optional SUT Adaptations

Implementations of certain network protocols, such as DTLS, may retransmit packets or prematurely terminate interactions if a response is not received within a predefined timeout. This can lead to timing-based differences, which are beyond this work’s scope as they are hard to reproduce. We therefore suppressed the time-based behavior of implementations, either by appropriate settings via the SUT’s library API or, when not possible, by making minor changes to the SUT’s code to disable timeouts.

In security protocols, common cryptographic operations (e.g., encryption, hashing) are time-consuming for SE. To speed up SE, these operations may be disabled/simplified/avoided (e.g., by appropriate SUT settings or minor modifications in the SUT’s code). In DTLS, after the client and server agree on a shared key, subsequent exchanges are encrypted. To avoid this encryption, we configured the parties to not use encryption when communicating.

4 EVALUATION

In this section, we evaluate our technique across several widely used DTLS implementations. Through this evaluation, we aim to answer the following questions:

- Q.1 How effective is our technique at finding bugs compared to the state-of-the-art in SE-based protocol testing when considering DTLS server implementations?
- Q.2 How does the required testing effort of our technique compare to the state-of-the-art, and how can it be optimized?
- Q.3 Can our technique identify previously unknown bugs in the latest versions of DTLS implementations?
- Q.4 Can our technique detect regressions?

Section 4.1 describes the evaluation setup. Section 4.2 answers Q.1 and Q.2, while Sections 4.3 and 4.4 answer Q.3 and Q.4, respectively.

4.1 Evaluation Setup

We compare our approach to that of Asadian et al. [2], which is recent, it also employs symbolic execution for protocol testing, and is similar also in the sense that it targets DTLS version 1.2. We target DTLS servers, for which we have developed test harnesses. We tested the server implementations of Mbed TLS, OpenSSL, wolfSSL and TinyDTLS, which were also analyzed by Asadian et al. [2]. We consider the original versions/commits analyzed by Asadian et al. [2] to answer Q.1 and Q.2, and the latest commits at the time of writing to answer Q.3 and Q.4. Concretely, we tested servers of Mbed TLS versions 3.1.0 and 3.5.0, OpenSSL versions 3.0.2 and 3.2.0, wolfSSL versions 5.3.0 and 5.6.3, and TinyDTLS commits e4d547ce (old) and f1d47d5 (latest). We were able to use the exact same test harness for Mbed TLS, OpenSSL and wolfSSL across the two versions tested for each, and had to perform only minor changes to the test harness between the two commits considered for TinyDTLS. This suggests that once a test harness has been developed for a library, it can be reused across multiple versions of the library with only minor or no adaptations.

Table 1: Discovered bugs in various versions of DTLS server implementations. Each row lists the implementation version, a brief description of the bug and its resolution status. Clicking on the status leads to the corresponding GitHub issue used for reporting the bug. Descriptions highlighted with a cyan background and blue text indicate new bugs, while those marked in black text indicate rediscoveries of bugs previously reported by Asadian et al. [2]. Footnotes denote bugs that affect multiple versions. New SUTs tested in this work are shown in blue, while old SUTs tested also by Asadian et al. are in black.

SUT	Short Description of the Bug(s)	Status
TinyDTLS e4d547ce	Out-of-bounds pointers caused by a handshake message other than <i>ClientHello</i> whose message type field has been set to <i>ClientHello</i>	Fixed
	Out-of-bounds pointers caused by a <i>ChangeCipherSpec</i> message whose content type field has been set to <i>Alert</i>	Fixed
	Record sequence number wrapping in a response triggered by receiving a record with the maximum possible record sequence number [†]	Confirmed
TinyDTLS f1d47d5	Susceptibility to Denial-of-Service due to generating an alert instead of silently discarding a record of invalid content type	Fixed
wolfSSL 5.3.0	Non-conformance bug when the message sequence numbers in the initial <i>ClientHello</i> and <i>HelloVerifyRequest</i> differ	Fixed
	Non-conformance bug when the message sequence numbers in the second <i>ClientHello</i> and <i>ServerHello</i> differ	Fixed
	Non-conformance bug when a server incorrectly discards <i>ClientHello</i> messages with large record sequence numbers [‡]	Confirmed
wolfSSL 5.6.3	Non-conformance bug where the server fails to discard a <i>ClientKeyExchange</i> message containing a duplicate record sequence number	Fixed
	Non-conformance bug where a server proceeds with the interaction despite receiving <i>ClientHello</i> messages with a non-existing version	Fixed

[†] This bug also affects TinyDTLS f1d47d5.

[‡] This bug also affects wolfSSL 5.6.3.

4.2 Comparison with a Related Technique on DTLS Servers

To address Q.1 and Q.2, we compare our technique with the approach proposed by Asadian et al. [2], which uses symbolic execution to check the conformance of protocol implementations against specification-derived requirements. For a fair comparison, we apply both approaches to the same DTLS server implementations and versions evaluated by Asadian et al. [2].

Concretely, for each requirement tested by Asadian et al., we constructed a corresponding server-side lightweight monitor that makes symbolic only the field relevant to that requirement. We then performed SE using these monitors on the same DTLS servers, and collected the resulting Path-Output pairs. Instead of checking these executions against specification assertions, as done by Asadian et al., we performed pairwise comparisons of the Path-Output pairs produced by different implementations and analyzed the resulting discrepancies.

Using this setup, our approach successfully identified *all* bugs reported by Asadian et al. [2]. Furthermore, our approach uncovered two additional bugs that were missed by their specification-based technique. These results indicate that our approach is more effective at exposing deviations in implementation behavior.

All identified bugs are summarized in Table 1. We next explain the two additional bugs and discuss why they were not detected by the approach of Asadian et al.. We conclude this section by answering Q.2.

Record Sequence Number Wrap. TinyDTLS mishandles the second *ClientHello* received during a DTLS handshake if the (record) sequence number in this message is the maximum possible value. In this case, TinyDTLS responds with a *ServerHello* with the same sequence number, followed by *ServerHelloDone* with sequence number zero, indicating a wrap-around⁴. By doing this, TinyDTLS violates the DTLS 1.2 RFC [27, p. 9], according to which, DTLS implementations MUST either abandon an association or initiate a re-handshake before the sequence number wraps. TinyDTLS does neither.

⁴The server is supposed to increment the sequence number in each record it sends.

This bug was missed by the technique of Asadian et al. [2] because it only tested if DTLS server implementations correctly check uniqueness of sequence numbers, and not how they handle (rare) wrapping situations.

Handling Large Record Sequence Numbers. wolfSSL incorrectly responds to the first or second *ClientHello* of a DTLS handshake by discarding these messages if their encapsulating records have sequence numbers which exceed a certain large threshold. According to the DTLS 1.2 RFC [27, p. 9], records should only be discarded if their sequence numbers are large enough to potentially cause sequence number wrapping, which is not the case here. Therefore, wolfSSL’s behavior does not align with the RFC and can lead to interoperability problems with other DTLS implementations which accept large sequence numbers.

This bug was missed by the technique of Asadian et al. [2] because it stems from a corner case for which no test (monitor) was implemented to appropriately check the handling of large sequence numbers.

Regarding Q.2, this approach has the advantage that it does not require tests to be constructed for the entire specification, which is impractical for complex protocols such as DTLS, whose behavior is described across multiple RFC documents. Instead, discrepancies are identified automatically using lightweight monitors that merely designate selected protocol fields as symbolic.

In our approach, however, identified discrepancies must be inspected against the RFC to determine whether they correspond to actual bugs. By contrast, the specification-based approach of Asadian et al. [2] detects bugs automatically once a test harness and a (complete) set of specification-derived tests have been constructed. It is important to note that not all discrepancies indicate bugs, as the RFC may explicitly permit multiple compliant behaviors.

A limitation of our approach is that it may miss bugs that are exhibited in the same way by all tested implementations, although this can be mitigated by testing a larger and more diverse set of implementations. Both approaches also require the manual construction of a test harness of comparable complexity for each SUT.

In practice, the two approaches are complementary. The monitor-based infrastructure is shared across both techniques, making their combination natural. The specification-based approach of Asadian et al. [2] can automatically detect bugs using tests derived directly from the specification, while our approach can uncover new bugs and extend the existing test suite so that these bugs can subsequently be detected automatically in other implementations.

4.3 Effectiveness on Recent DTLS Implementations

To answer Q.3, we applied our technique to the latest versions of Mbed TLS 3.5.0, OpenSSL 3.2.0, TinyDTLS commit f1d47d5, and wolfSSL 5.6.3. Our technique identified bugs in two of these systems (TinyDTLS and wolfSSL), indicating that the approach is effective at uncovering previously unknown issues in modern DTLS implementations. Concretely, it discovered nine bugs, five of which were previously unknown, highlighted with blue text/cyan background in Table 1. All previously unknown bugs were reported to the corresponding developers and have since been fixed or confirmed. We proceed to analyze these bugs in detail.

Handling Invalid Content Type. TinyDTLS responds with a warning level alert to a *ClientKeyExchange* message encased in a record with the content type set to *alert*, instead of its expected value *handshake*. This goes against the RFC [27, p. 14], which specifies that in this case an implementation should simply discard the record, or must alternatively issue a fatal level alert. Subsequently, the RFC recommends against issuing fatal alerts, as it can make an implementation susceptible to DoS attacks. Consequently, TinyDTLS’s behavior is non-compliant and possibly susceptible to DoS attacks.

Handling Duplicate Record Sequence Numbers. wolfSSL accepts a *ClientKeyExchange* message even if its (record) sequence number duplicates that in the initial *ClientHello*. According to the DTLS 1.2 RFC [27, p. 13], implementations should check that records received have unique sequence numbers and should reject those that do not. wolfSSL fails to enforce this RFC requirement.

Handling Invalid Record Version. wolfSSL fails to validate the record version for records containing *ClientHello* messages. According to the DTLS 1.2 RFC [27, p. 8], the version field indicates the version of the protocol being used. The TLS 1.2 RFC [11, p. 88], on which DTLS 1.2 is based, emphasizes the importance of validating the first byte of the version field. These observations indicate that wolfSSL does not fully comply with the specification.

4.4 Effectiveness in Detecting Regressions

To evaluate the effectiveness of our technique in detecting regressions, we conducted a controlled experiment using TinyDTLS. Specifically, we focused on a previously identified vulnerability documented in Issue #74 of the TinyDTLS repository. This vulnerability entails bypassing the server’s cookie check on the second *ClientHello* message in a handshake, by using an invalid version in the record encapsulating this message. TinyDTLS will then proceed to the next phase of the handshake, becoming susceptible to DoS

attacks.⁵ This vulnerability has since been fixed and does not affect the latest version.

Our investigation indicates that the vulnerability was present since the first commit. We thus artificially reintroduced the vulnerability onto the latest version of TinyDTLS, which we used as baseline since it properly managed *ClientHello* messages with invalid version numbers. We then applied our technique to compare the behavior of the original (unmodified) version against that of the modified (buggy) version. In this experiment, the version field of the initial *ClientHello* message was made symbolic. Symbolic execution explored seven execution paths for each version of the implementation. In both cases, SE completed in under ten seconds. The comparison of the resulting Path-Output pairs revealed a single discrepancy, which was detected within one second. Inspection of this discrepancy directly exposed the reintroduced vulnerability in the modified version.

These results demonstrate that the proposed technique can efficiently detect regressions with minimal exploration effort and low analysis overhead. This experiment therefore provides an affirmative answer to Q.4.

5 RELATED WORK

Differential testing has been widely applied to uncover semantic bugs in network protocol implementations by comparing multiple implementations under the same inputs. Among these approaches, differential symbolic execution, introduced by Person et al. [24], extended SE to systematically identify and analyze behavioral differences across versions. However, this technique has been applied mostly to stateless systems.

ParDiff [36] introduces a static differential analysis framework that extracts finite-state machines (FSMs) from the message parsing code of different protocol implementations and applies bisimulation with SMT solving to identify semantic inconsistencies between the FSMs. Chau et al. [7, 8] employ differential symbolic execution to classify certificate chains as valid or invalid, generating path constraints for each outcome. These constraints are then compared across implementations to identify and analyze discrepancies. In contrast to these approaches, which focus on isolated components of an implementation, our work performs dynamic differential SE of the entire protocol implementation, enabling the detection of specification violations during stateful protocol interactions.

Other approaches combine symbolic execution with differential testing by synthesizing concrete test cases and running them to detect discrepancies. RFCcert [9] extracts rule constraints from the relevant RFCs, uses SE to construct certificates that satisfy or violate each rule, and then differentially tests multiple X.509 validators to pinpoint specification violations. SymTCP [34] applies selective SE to an end host’s TCP stack to synthesize packets that reach key accept or drop paths, then sends them through black-box DPI middleboxes and the end host; any disagreement reveals insertion or evasion opportunities that desynchronize DPI state. Both methods use symbolic execution to generate concrete test cases,

⁵The cookie field in the second *ClientHello* should duplicate the cookie in the server’s *HelloVerifyRequest* response to the first *ClientHello*. This mechanism prevents DoS attacks as it ensures an attacker is unable to exhaust the resources of server by flooding it with many *ClientHello* messages which prompt the server to start as many computationally-expensive handshakes.

which are run on the SUTs to find discrepancies. By contrast, we use symbolic execution to identify discrepancies by systematically cross-examining path conditions of the SUTs.

Specification-based testing checks an implementation against an oracle derived from the specification. Several approaches [1, 2, 29–31] propose incorporating SE to improve the detection power of specification-based testing. KleeNet [30] introduces a technique that integrates KLEE into the TinyOS platform for testing sensor network applications. Using KleeNet, correctness properties can be tested by manually adding assertions to the SUT’s codebase. A follow-up work [29] extended KleeNet with a simulated network model, enabling SE of distributed applications across multiple nodes. Packet loss and node failures can be represented as symbolic variables, allowing SE to explore different error scenarios.

SymbexNet [31] focuses on checking specification-derived requirements using symbolic execution. It performs SE on both single- and multi-packet exchanges to generate high-coverage concrete test packets. These packets are then replayed on the unmodified implementation, and the resulting input-output streams are checked against requirements derived from the protocol specification. SE can overlook inputs that violate requirements when those inputs follow the same code path in the SUT as compliant ones. Asadian et al. [1] address this issue by embedding assertions and assumptions directly into the SUT’s code, guiding the symbolic execution engine toward code paths exercised by packet sequences that may violate protocol requirements. Subsequent work [2] moves the requirements into monitors that reside outside the SUT. These monitors provide a uniform mechanism for validating protocol requirements and reduce the need for extensive SUT changes.

Although these specification-based techniques are effective when the requirements are accurately captured in an oracle, they are inherently incomplete. Any requirement not explicitly encoded in the oracle will remain unchecked, as observed in §4.

In addition to white-box approaches, black-box techniques have been used to evaluate protocol implementations against their specifications. Protocol state fuzzing, for example, applies model learning [25, 33] to infer state machine models of implementations, often in the form of Mealy machines. This technique has been extensively applied to network and IoT protocols, including TLS [15], TCP [13, 14], SSH [16], OpenVPN [10], IPSec [17], QUIC [12, 26], and EDHOC [28].

6 CONCLUSIONS

In this paper, we presented a differential symbolic execution approach for testing the stateful behavior of network protocol implementations without relying on an explicitly specified oracle. Our approach performs SE of the SUTs, treating as symbolic selected fields in input packets exchanged during an interaction, and recording the SUT’s responses to these packets. It automatically processes the results of SE to identify inputs that satisfy path conditions of both SUTs while prompting different responses. These discrepancies are analyzed for conformance.

We evaluated our approach on DTLS server implementations of four widely-used libraries, and demonstrated its effectiveness. In short, our approach rediscovered all four bugs previously identified by the state-of-the-art specification-based testing approaches,

while also identifying two new bugs that those techniques missed. Our approach also identified four new bugs in the latest versions of the libraries tested. In total, we identified nine distinct bugs across versions, five of which were previously unknown and later confirmed or fixed by maintainers. We also showed that our approach can be effective to detect regressions in a cross-version setting. While clearly outperforming specification-based testing in our evaluation, it is worth stressing that our approach is a complement to specification-based testing, rather than a replacement. Unlike specification-based testing, our approach’s effectiveness does not hinge on the availability of an oracle encoding all the specification’s requirements. Yet our approach is also not fully automatic, requiring manual inspection of discrepancies. Future work can explore combining our approach with specification-based testing, such as Asadian et al., to increase automation.

REFERENCES

- [1] Hooman Asadian, Paul Fiterău-Broștean, Bengt Jonsson, and Konstantinos Sagonas. 2022. Applying Symbolic Execution to Test Implementations of a Network Protocol Against its Specification. In *IEEE Conference on Software Testing, Verification and Validation (ICST 2022)*. IEEE, 70–81. <https://doi.org/10.1109/ICST53961.2022.00019>
- [2] Hooman Asadian, Paul Fiterău-Broștean, Bengt Jonsson, and Konstantinos Sagonas. 2024. Monitor-based Testing of Network Protocol Implementations Using Symbolic Execution. In *Proceedings of the 19th International Conference on Availability, Reliability and Security (ARES '24)*. ACM, 1–12. <https://doi.org/10.1145/3664476.3664521>
- [3] Benjamin Beurdouche, Karthikeyan Bhargavan, Antoine Delignat-Lavaud, Cédric Fournet, Markulf Kohlweiss, Alfredo Pirotti, Pierre-Yves Strub, and Jean Karim Zinzindohoue. 2015. A Messy State of the Union: taming the Composite State Machines of TLS. In *IEEE Symposium on Security and Privacy*. IEEE.
- [4] Hanno Böck, Juraj Somorovsky, and Craig Young. 2018. Return Of Bleichenbacher’s Oracle Threat (ROBOT). In *27th USENIX Security Symposium (USENIX Security 18)*. USENIX Association, Baltimore, MD, 817–849. <https://www.usenix.org/conference/usenixsecurity18/presentation/bock>
- [5] Chad Brubaker, Suman Jana, Baishakhi Ray, Sarfraz Khurshid, and Vitaly Shmatikov. 2014. Using Frankencerts for Automated Adversarial Testing of Certificate Validation in SSL/TLS Implementations. In *2014 IEEE Symposium on Security and Privacy*. 114–129. <https://doi.org/10.1109/SP.2014.15>
- [6] Cristian Cadar, Daniel Dunbar, and Dawson Engler. 2008. KLEE: Unassisted and Automatic Generation of High-Coverage Tests for Complex Systems Programs. In *Proceedings of the 8th USENIX Conference on Operating Systems Design and Implementation (San Diego, California) (OSDI '08)*. USENIX Association, Berkeley, CA, USA, 209–224. <http://dl.acm.org/citation.cfm?id=1855741.1855756>
- [7] Sze Yiu Chau, Omar Chowdhury, Md. Endadul Hoque, Huangyi Ge, Aniket Kate, Cristina Nita-Rotaru, and Ninghui Li. 2017. SymCerts: Practical Symbolic Execution for Exposing Noncompliance in X.509 Certificate Validation Implementations. In *2017 IEEE Symposium on Security and Privacy (San Jose, CA, USA) (SP 2017)*. IEEE Computer Society, 503–520. <https://doi.org/10.1109/SP.2017.40>
- [8] Sze Yiu Chau, Moosa Yahyazadeh, Omar Chowdhury, Aniket Kate, and Ninghui Li. 2019. Analyzing Semantic Correctness with Symbolic Execution: A Case Study on PKCS#1 v1.5 Signature Verification. In *26th Annual Network and Distributed System Security Symposium (San Diego, CA, USA) (NDSS 2019)*. The Internet Society. <https://doi.org/10.14722/ndss.2019.23430>
- [9] Chu Chen, Cong Tian, Zhenhua Duan, and Liang Zhao. 2018. RFC-directed differential testing of certificate validation in SSL/TLS implementations. In *Proceedings of the 40th International Conference on Software Engineering (Gothenburg, Sweden) (ICSE 2018)*, Michel Chaudron, Ivica Crnkovic, Marsha Chechik, and Mark Harman (Eds.). ACM, 859–870. <https://doi.org/10.1145/3180155.3180226>
- [10] Lesly-Ann Daniel, Erik Poll, and Joeri de Ruiter. 2018. Inferring OpenVPN State Machines Using Protocol State Fuzzing. In *IEEE European Symposium on Security and Privacy (EuroS&P) Workshops (London, U.K.)*. IEEE, 11–19. <https://doi.org/10.1109/EuroSPW.2018.00009>
- [11] Tim Dierks and Eric Rescorla. 2008. The Transport Layer Security TLS Protocol Version 1.2. RFC 5246. <http://www.rfc-editor.org/rfc/rfc5246.txt>
- [12] Tiago Ferreira, Harrison Brewton, Loris D’Antoni, and Alexandra Silva. 2021. Prognosis: Closed-box Analysis of Network Protocol Implementations. In *ACM SIGCOMM 2021 Conference*. ACM, 762–774. <https://doi.org/10.1145/3452296.3472938>
- [13] Paul Fiterău-Broștean and Falk Howar. 2017. Learning-Based Testing the Sliding Window Behavior of TCP Implementations. In *Critical Systems: Formal Methods*

- and Automated Verification - Joint 22nd International Workshop on Formal Methods for Industrial Critical Systems and 17th International Workshop on Automated Verification of Critical Systems, FMICS-AVoCS 2017. LNCS, Vol. 10471. Springer, 185–200. https://doi.org/10.1007/978-3-319-67113-0_12
- [14] Paul Fiterău-Broștean, Ramon Janssen, and Frits W. Vaandrager. 2016. Combining Model Learning and Model Checking to Analyze TCP Implementations. In *Computer Aided Verification - 28th International Conference, CAV 2016, Proceedings, Part II (LNCS, Vol. 9780)*. Springer, 454–471. https://doi.org/10.1007/978-3-319-41540-6_25
- [15] Paul Fiterău-Broștean, Bengt Jonsson, Konstantinos Sagonas, and Fredrik Täquist. 2022. DTLS-Fuzzer: A DTLS Protocol State Fuzzer. In *15th IEEE International Conference on Software Testing, Verification and Validation (ICST 2022)*. IEEE Computer Society.
- [16] Paul Fiterău-Broștean, Toon Lenaerts, Joeri de Ruiter, Erik Poll, Frits W. Vaandrager, and Patrick Verleg. 2017. Model Learning and Model Checking of SSH Implementations. In *Proceedings of the 24th ACM SIGSOFT International SPIN Symposium on Model Checking of Software* (Santa Barbara, CA, USA) (SPIN 2017). ACM, 142–151. <https://doi.org/10.1145/3092282.3092289>
- [17] Jiaxing Guo, Chunxiang Gu, Xi Chen, and Fushan Wei. 2019. Model Learning and Model Checking of IPsec Implementations for Internet of Things. *IEEE Access* 7 (Nov. 2019), 171322–171332. <https://doi.org/10.1109/ACCESS.2019.2956062>
- [18] Samuel Jero, Maria Leonor Pacheco, Dan Goldwasser, and Cristina Nita-Rotaru. 2019. Leveraging textual specifications for grammar-based fuzzing of network protocols. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 33. 9478–9483. <https://doi.org/10.1609/aaai.v33i01.33019478>
- [19] Siva Kesava Reddy Kakarla, Ryan Beckett, Todd Millstein, and George Varghese. 2022. SCALE: Automatically Finding RFC Compliance Bugs in DNS Nameservers. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*. USENIX Association, Renton, WA, 307–323. <https://www.usenix.org/conference/nsdi22/presentation/kakarla>
- [20] James C. King. 1976. Symbolic Execution and Program Testing. *Commun. ACM* 19, 7 (July 1976), 385–394. <https://doi.org/10.1145/360248.360252>
- [21] William M McKeeman. 1998. Differential testing for software. *Digital Technical Journal* 10, 1 (1998), 100–107.
- [22] Kenneth L. McMillan and Lenore D. Zuck. 2019. Formal Specification and Testing of QUIC. In *Proceedings of the ACM Special Interest Group on Data Communication* (Beijing, China) (SIGCOMM 2019). ACM, 227–240. <https://doi.org/10.1145/3341302.3342087>
- [23] Bodo Möller, Thai Duong, and Krzysztof Kotowicz. 2014. This POODLE bites: exploiting the SSL 3.0 fallback. <https://www.openssl.org/~bodo/ssl-poodle.pdf>
- [24] Suzette Person, Matthew B. Dwyer, Sebastian Elbaum, and Corina S. Pasareanu. 2008. Differential symbolic execution. In *Proceedings of the 16th ACM SIGSOFT International Symposium on Foundations of Software Engineering* (Atlanta, Georgia) (SIGSOFT '08/FSE-16). Association for Computing Machinery, New York, NY, USA, 226–237. <https://doi.org/10.1145/1453101.1453131>
- [25] Harald Raffelt, Maik Merten, Bernhard Steffen, and Tiziana Margaria. 2009. Dynamic testing via automata learning. *Int. J. Softw. Tools Technol. Transf.* 11, 4 (2009), 307–324. <https://doi.org/10.1007/s10009-009-0120-7>
- [26] Abdullah Rasool, Greg Alpár, and Joeri de Ruiter. 2019. State machine inference of QUIC. (2019). arXiv preprint arXiv:1903.04384.
- [27] Eric Rescorla and Nagendra Modadugu. 2012. Datagram Transport Layer Security Version 1.2. RFC 6347, 32 pages. <http://www.rfc-editor.org/rfc/rfc6347.txt>
- [28] Konstantinos Sagonas and Thanasis Typaldos. 2023. EDHOC-Fuzzer: An EDHOC protocol state fuzzer. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis* (Seattle, WA, USA) (ISSTA '23). René Just and Gordon Fraser (Eds.). ACM, New York, NY, USA, 1495–1498. <https://doi.org/10.1145/3597926.3604922>
- [29] Raimondas Sasnauskas, Olaf Landsiedel, Muhammad Hamad Alizai, Carsten Weise, Stefan Kowalewski, and Klaus Wehrle. 2010. KleeNet: Discovering Insidious Interaction Bugs in Wireless Sensor Networks Before Deployment. In *Proceedings of the 9th International Conference on Information Processing in Sensor Networks* (Stockholm, Sweden) (IPSN 2010). ACM, 186–196. <https://doi.org/10.1145/1791212.1791235>
- [30] Raimondas Sasnauskas, Jó Ágila Bitsch Link, Muhammad Hamad Alizai, and Klaus Wehrle. 2008. KleeNet: automatic bug hunting in sensor network applications. In *Proceedings of the 6th International Conference on Embedded Networked Sensor Systems* (Raleigh, NC, USA) (SenSys 2008). ACM, 425–426. <https://doi.org/10.1145/1460412.1460485>
- [31] JaeSeung Song, Cristian Cadar, and Peter R. Pietzuch. 2014. SYMBEXNET: Testing Network Protocol Implementations with Symbolic Execution and Rule-Based Specifications. *IEEE Trans. Software Eng.* 40, 7 (2014), 695–709. <https://doi.org/10.1109/TSE.2014.2323977>
- [32] W. Richard Stevens. 1990. *Unix network programming*. Prentice Hall.
- [33] Frits W. Vaandrager. 2017. Model learning. *Commun. ACM* 60, 2 (2017), 86–95. <https://doi.org/10.1145/2967606>
- [34] Zhongjie Wang, Shitong Zhu, Yue Cao, Zhiyun Qian, Chengyu Song, Srikanth V Krishnamurthy, Kevin S Chan, and Tracy D Braun. 2020. SymTCP: Eluding Stateful Deep Packet Inspection with Automated Discrepancy Discovery. In *NDSS (NDSS 2020)*. <https://doi.org/10.14722/ndss.2020.24083>
- [35] Xuejun Yang, Yang Chen, Eric Eide, and John Regehr. 2011. Finding and understanding bugs in C compilers. In *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation* (San Jose, California, USA) (PLDI '11). Association for Computing Machinery, New York, NY, USA, 283–294. <https://doi.org/10.1145/1993498.1993532>
- [36] Mingwei Zheng, Qingkai Shi, Xuwei Liu, Xiangzhe Xu, Le Yu, Congyu Liu, Guannan Wei, and Xiangyu Zhang. 2024. ParDiff: Practical Static Differential Analysis of Network Protocol Parsers. *Proceedings of the ACM on Programming Languages* 8 (04 2024), 1208–1234. <https://doi.org/10.1145/3649854>